DB11

Informal Design Guidelines for Relation Schemas

> **좋은 Relatinoal schema인지 판단하기 위한 비형식적 가이드라인**

왜 Informal?

- **사람의 의미 해석에 기반**하기 때문이다.

Desinging a relational database

> **주어진 Application에 대해 좋은 Table schema의 집합을 정의하는 과정**

- 각 Table에 포함되는 Attribute를 결정하는 과정이다.

Good table schema 판단 기준

> **하나의 Table schema가 좋은지 판단하는 기준**

1. Semantics of attributes

> **Table schema를 Determining할 때, 한 Table의 속한 Attribute들은 현실 세계에서 의미적으로 서로 연관되어 있어야 한다.**

Guideline 1

> **각 Tuple은 정확히 하나의 ENTITY Instance나 RELATIONSHIP Instance에 대응**되어야 한다.

- 여러 ENTITY나 RELATIONSHIP의 attribute를 하나의 Relation으로 결합하지 않는다.
- Instance: Tuple 하나

2. Redundant information in tuples

저장 공간을 줄이기 위해 Redundant information을 제거한다.

- 저장 공간을 줄이는 것은 Table schema design의 주요 목적 중 하나이다.
- 서로 다른 ENTITY에 대한 정보를 하나에 결합하기 때문에 발생한다.

1:N 또는 N:M **Relationship에 참여하는 서로 다른 두 Entity를 하나의 Entity로 결합하면 Redundancy**가 생긴다.

- 1:N

Redundancy

EMP_DEPT

Ename	Ssn	Bdate	Address	Dnumber	Dname	Dmgr_ssn
Smith, John B.	123456789	1965-01-09	731 Fondren, Houston, TX	5	Research	333445555
Wong, Franklin T.	333445555	1955-12-08	638 Voss, Houston, TX	5	Research	333445555
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle, Spring, TX	4	Administration	987654321
Wallace, Jennifer S.	987654321	1941-06-20	291 Berry, Bellaire, TX	4	Administration	987654321
Narayan, Ramesh K.	666884444	1962-09-15	975 FireOak, Humble, TX	5	Research	333445555
English, Joyce A.	453453453	1972-07-31	5631 Rice, Houston, TX	5	Research	333445555
Jabbar, Ahmad V.	987987987	1969-03-29	980 Dallas, Houston, TX	4	Administration	987654321
Borg, James E.	888665555	1937-11-10	450 Stone, Houston, TX	1	Headquarters	888665555

- N:M

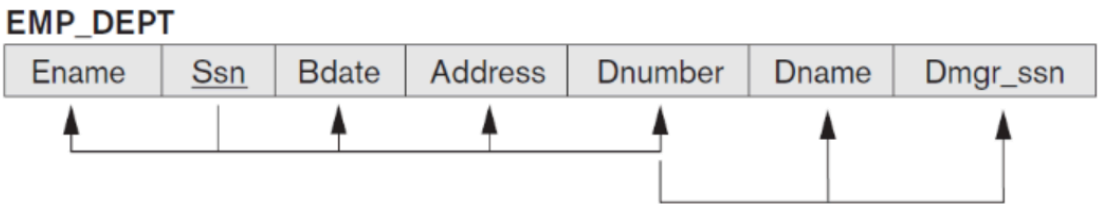
Redundancy Redundancy

EMP_PROJ

Ssn	Pnumber	Hours	Ename	Pname	Plocation
123456789	1	32.5	Smith, John B.	ProductX	Bellaire
123456789	2	7.5	Smith, John B.	ProductY	Sugarland
666884444	3	40.0	Narayan, Ramesh K.	ProductZ	Houston
453453453	1	20.0	English, Joyce A.	ProductX	Bellaire
453453453	2	20.0	English, Joyce A.	ProductY	Sugarland
333445555	2	10.0	Wong, Franklin T.	ProductY	Sugarland
333445555	3	10.0	Wong, Franklin T.	ProductZ	Houston
333445555	10	10.0	Wong, Franklin T.	Computerization	Stafford
333445555	20	10.0	Wong, Franklin T.	Reorganization	Houston
999887777	30	30.0	Zelaya, Alicia J.	Newbenefits	Stafford
999887777	10	10.0	Zelaya, Alicia J.	Computerization	Stafford
987987987	10	35.0	Jabbar, Ahmad V.	Computerization	Stafford
987987987	30	5.0	Jabbar, Ahmad V.	Newbenefits	Stafford
987654321	30	20.0	Wallace, Jennifer S.	Newbenefits	Stafford
987654321	20	15.0	Wallace, Jennifer S.	Reorganization	Houston
888665555	20	Null	Borg, James E.	Reorganization	Houston

Update Anomaly

> **Redundancy로 인하여 Insertion / Deletion / Modification이 기대대로 실행되지 못하는 경우**



1. **Insertion anomalies**

   EX) `EMP_DEPT` 테이블에서 아직 할당된 Employee가 없는 Department를 Insert하기 어려운 경우

   - EMP_DEPT의 PK가 Ssn이라서 Insert가 불가능하다.

- Table이 잘못 설계되어 Employee / Department의 두 테이블로 나뉘지 않았기 때문이다.

2. **Deletion anomalies**

   EX) `EMP_DEPT` 테이블에서 마지막 Employee를 삭제하는 경우 삭제할 수 없다.

   - EMP_DEPT의 PK가 Ssn이라서 Delete가 불가능하다.
   - Table이 잘못 설계되어 Employee / Department의 두 테이블로 나뉘지 않았기 때문이다.

3. **Modification anomalies**

   EX) `EMP_DEPT` 테이블에서 Department의 Manager가 변경되면 해당 Department에 속한 Employee의 Department manager 정보를 전부 수정해야 한다.

   - 기대했던 동작: 한 번만 수정
   - Table이 잘못 설계되어 Employee / Department의 두 테이블로 나뉘지 않았기 때문이다.

Guideline 2

> **Update anomaly가 존재하지 않도록 하는 Base relatoin schema를 Design**

- Base relation shcema = CREATE TABLE로 생성된 Table

불가피하게 Update Anomalies가 존재한다면?

1. 어떤 Anomalies가 존재하는지 명시
2. Program에서 해당 Anomalies가 존재하지 않도록 처리

3. NULL values in tuples

NULL: Unknown / Unavailable → Not Applicable

- 저장 공간 낭비
- Ambiguous한 의미
- 연산자를 통한 비교 불가
- Aggregate function을 사용하기 어렵다.
    - Aggregate function은 NULL 값을 무시하고 계산
    - NULL 값이 압도적으로 많다면 Aggregate function을 적용해도 의미가 없다.

Guideline 3

> **NULL 값을 최대한 가지지 않도록 Base relation schema를 Design**

- 자주 NULL이 될 것 같은 Attribute는 Base relation의 Attribute로 사용하지 않는다.

불가피하게 NULL이 존재한다면?

- **NULL 값을 갖는 것이 예외적인 상황**이고 대부분 값이 존재하도록 해야 한다.
- NULL 값이 존재하는 Attribute를 따로 다루는 **새로운 Relation을 생성**한다.
    - 해당 Attribute에서 NULL 값이 아닌 실제로 존재하는 값만 존재하도록 새로운 Relation을 생성한다.

NULL 값을 각기 다르게 처리하는 여러 방법을 보자.

If only 10 percent of employees have their own individual offices

1. **Employee table에** `Office_num` **이라는 Attribute를 추가**
    - 90%의 Tuple이 NULL이 된다.

2. `EMP_OFFICES (Essn, Office_number)` **라는 새로운 Relation을 생성하는 방법**

- Office를 갖는 Employee만 위 Relation에 추가되기 때문에 NULL이 존재하지 않는다.

4. Spurious tuples

Spurious tuple: 원래의 의미 있는 정보에는 존재하지 않았지만, **JOIN 결과에서 인위적으로 생성된 잘못된 tuple**

- NATURAL JOIN에 의해 주로 생성된다.
  - NATURAL JOIN으로 인해 KEY Attribute가 아닌 Attribute를 기준으로 JOIN이 되면 Spurious tuple이 생성될 수 있다.
- NATURAL JOIN이 아니더라도 **JOIN Attribute가 Key attribute가 아니라면 발생할 수 있다.**
- Relation이 잘못 설정되어 있으면 NATURAL JOIN에서 쉽게 Spurious tuple이 생성된다.

EMP_LOCS

Ename	Plocation
Smith, John B.	Bellaire
Smith, John B.	Sugarland
Narayan, Ramesh K.	Houston
English, Joyce A.	Bellaire
English, Joyce A.	Sugarland
Wong, Franklin T.	Sugarland
Wong, Franklin T.	Houston
Wong, Franklin T.	Stafford
Zelaya, Alicia J.	Stafford
Jabbar, Ahmad V.	Stafford
Wallace, Jennifer S.	Stafford
Wallace, Jennifer S.	Houston
Borg, James E.	Houston

EMP_PROJ1

Ssn	Pnumber	Hours	Pname	Plocation
123456789	1	32.5	ProductX	Bellaire
123456789	2	7.5	ProductY	Sugarland
666884444	3	40.0	ProductZ	Houston
453453453	1	20.0	ProductX	Bellaire
453453453	2	20.0	ProductY	Sugarland
333445555	2	10.0	ProductY	Sugarland
333445555	3	10.0	ProductZ	Houston
333445555	10	10.0	Computerization	Stafford
333445555	20	10.0	Reorganization	Houston
999887777	30	30.0	Newbenefits	Stafford
999887777	10	10.0	Computerization	Stafford
987987987	10	35.0	Computerization	Stafford
987987987	30	5.0	Newbenefits	Stafford
987654321	30	20.0	Newbenefits	Stafford
987654321	20	15.0	Reorganization	Houston
888665555	20	NULL	Reorganization	Houston

- 위 상황에서 NATURAL JOIN을 하면 `Plocation` 이 JOIN Attribute가 된다.
  - `Plocation` 은 Key attribute가 아니다.

- 이 경우 `English, Joyce A.'의 Ssn이 123456789가 아님에도 Ssn이 123456789인 Tuple이 생성된다.
    - Spurious tuple

Guideline 4

> **JOIN Attribute가 PK와 FK가 되도록 Base relation schema를 정의해야 한다.**

- **서로 다른 두 Relation에서 겹치는 Attribute는 서로 대응되는 PK-FK**만 존재하도록 한다.

Functional Dependencies

> **Relational schema를 분석하기 위한 형식적인 툴**

- key와 함께 normal form을 정의하는 데 사용된다.

$$X \rightarrow Y$$

- X: Relation schema R에 속하는 Attribute의 집합
- Y: Relation schema R에 속하는 Attribute의 집합
    - X와 Y는 **반드시 같은 테이블 R의 attribute**
- X **Functionally determine** Y
- Y **is functionally dependent on** X

X가 결정되었을 때, Y가 유일하게 결정되는 것이다.

$$t_1[X] = t_2[X] \rightarrow t_1[Y] = t_2[Y]$$

City → Province와 같은 **현실 세계의 Constraints에서 유도**된다.

Properties

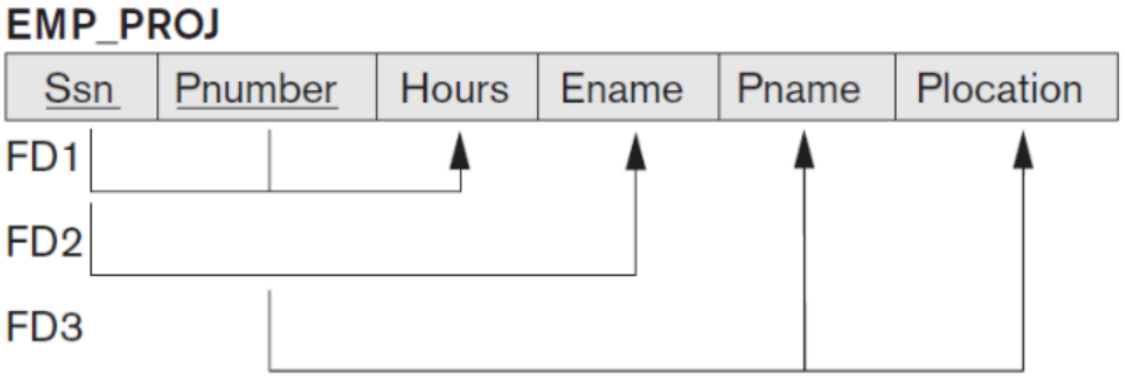
1. **X가 R의 Candidate key라면?**
   - R의 모든 가능한 Attribute의 Subset Y에 대해 X → Y이다.

2. **X → Y일 때, 반드시 Y → X인 것은 아니다.**
   - 그럴 수도 있고 아닐 수도 있다.

3. **FD는 Table schema R의 성질이다**.
   - FD는 특정 table instance r의 성질이 아니다.
   - FD는 R로부터 생성될 수 있는 **모든 가능한 instance $r(R)$에 대해 항상 성립**해야 한다.
     - 모든 가능한 경우에 대해 FD가 성립해야 한다.

4. **Functional Dependency를 정의하고 Table instance가 그것을 따르도록 하는 것이다**
   - Database가 정의되기 전에 명시적으로 Define되어야 한다.
   - 주어진 Instance 하나만 보고 FD를 자동으로 추론할 수 없다.
     - Instance를 보고 추론하는 것이 아니다.

Example



세 가지 FD가 존재한다고 할 수 있다.

- 실제로는 더 많이 있을 수는 있지만 사전에 3개의 FD를 정의했다고 가정하자.

a. Ssn $\rightarrow$ Ename

b. Pnumber $\rightarrow$ {Pname, Plocation}

c. {Ssn, Pnumber} $\rightarrow$ Hours

Inference Rule

> **이미 알고 있는 FD로부터 새로운 FD를 추론하는데 사용되는 규칙**

F: Table schema R에 대해 **미리 정의되어있는 Functional dependency의 집합**

F^+: **F로부터 추론할 수 있는 모든 Functional dependency의 집합**

- F를 포함한다.
- **The closure of F**

- Inference rule을 더 이상 적용해도 새로운 FD가 추가되지 않는 최종 상태의 집합

Amstrong's inference rules

Sound & Complete

- **Sound:** 잘못된 Rule이 없다
- **Complete:** 이 세 가지 Rule으로 F^+를 구할 수 있다.

IR1: Reflective rule

$$\text{if } X \supseteq Y, \text{then } X \to Y$$

IR2: Augmentation rule

$$X \to Y \models XZ \to YZ = X, Z \to Y, Z$$

IR3: Transitive rule

$$X \to Y, Y \to Z \models X \to Z$$

Sound 증명

- $t_1[X] = t_2[X] \to t_1[Y] = t_2[Y]$ 를 이용한다.

IR1 (Reflexive): $Y \subseteq X \Rightarrow X \to Y$ 가 sound

임의의 관계 r에서 두 튜플 t_1, t_2가 $t_1[X] = t_2[X]$ 라고 하자.
그런데 $Y \subseteq X$ 이므로 Y의 모든 속성은 X 안에 포함됨.
따라서 $t_1[Y] = t_2[Y]$.
즉 $r \models X \to Y$. ✅

🛠️ 확대 규칙 (IR2) 증명 과정 (수정 완료)

규칙: $\{X \to Y\} \models XZ \to YZ$

단계	논리적 흐름 (수식)	간단한 설명
1. 시작 가정	임의의 두 튜플 t_1, t_2에 대해 $t_1[XZ] = t_2[XZ]$ 라고 가정한다.	XZ에 대해 값이 같다.
2. 분해	$t_1[XZ] = t_2[XZ]$는 다음을 의미한다: $t_1[X] = t_2[X]$ **그리고** $t_1[Z] = t_2[Z]$.	X와 Z 각각에 대해 값이 같다.
3. 규칙 적용	전제 $X \to Y$와 $t_1[X] = t_2[X]$로부터 $t_1[Y] = t_2[Y]$가 유도된다.	$X \to Y$ 때문에 Y에 대해서도 값이 같다.
4. 결합	$t_1[Y] = t_2[Y]$와 $t_1[Z] = t_2[Z]$를 결합하여 $t_1[YZ] = t_2[YZ]$가 성립한다.	Y와 Z 모두 값이 같으니, YZ에 대해서도 값이 같다.
5. 최종 결론	시작 가정 $t_1[XZ] = t_2[XZ]$에서 최종적으로 $t_1[YZ] = t_2[YZ]$를 유도했으므로, $XZ \to YZ$가 성립한다.	$XZ \to YZ$가 유효하다.

Sheets로 내보내기

IR3 (Transitive): $X \to Y,\ Y \to Z \Rightarrow X \to Z$ 가 sound

$r \models X \to Y$ 그리고 $r \models Y \to Z$라 하자.
$t_1[X] = t_2[X] \Rightarrow t_1[Y] = t_2[Y] \Rightarrow t_1[Z] = t_2[Z]$.
따라서 $r \models X \to Z$. ✅

Other inference rules

IR1 ~ IR3를 이용하여 증명할 수 있다.

IR4: Decomposition rule

$$X \to YZ \models X \to Y, X \to Z$$

IR4. Decomposition rule 증명

목표:

$$X \to YZ \vdash X \to Y \quad \text{and} \quad X \to Z$$

(1) $X \to Y$ 유도

1. 주어짐: $X \to YZ$
2. IR1 (Reflexive): $YZ \to Y$ (왜냐면 $Y \subseteq YZ$)
3. IR3 (Transitive): $X \to YZ$ 와 $YZ \to Y$ 로부터

$$X \to Y$$

(2) $X \to Z$ 유도

1. 주어짐: $X \to YZ$
2. IR1: $YZ \to Z$ (왜냐면 $Z \subseteq YZ$)
3. IR3: $X \to YZ$ 와 $YZ \to Z$ 로부터

$$X \to Z$$

IR5: Union rule

$$X \to Y, X \to Z \models X \to YZ$$

1. 주어짐: $X \to Y$
2. IR2 (Augmentation) with Z:

$$XZ \to YZ$$

3. 주어짐: $X \to Z$
4. IR2 with X:

$$X \to Z \;\Rightarrow\; XX \to ZX$$

근데 집합으로 보면 $XX = X$, $ZX = XZ$ 이므로

$$X \to XZ$$

5. IR3 (Transitive): $X \to XZ$ 와 $XZ \to YZ$ 로부터

$$X \to YZ$$

✅ 따라서

$$X \to Y,\; X \to Z \vdash X \to YZ$$

IR6: Pseudo-transitive rule

$$X \to Y, WY \to Z \models WX \to Z$$

- Augmentation rule에 의해 $WX \to WY$
- Transitive rule에 의해 $WX \to Z$

Systematic Way to Deterine Additional FDs

X: FD $F : X \rightarrow Y$에서의 X의 Attribute의 집합

X^+: FD F에서 X에 의해 **Functionally determined되는 Attribute의 집합**

- **X의 Closure**라고도 한다.
- 각 X에 대해 Inference rule을 반복 적용하여 얻을 수 있다.
- Candidate key의 Closure는 항상 Table에 속한 Attribute 전체이다.

Algorithm

각 X에 대해 아래 알고리즘을 반복한다.

$X^+ := X;$
repeat
    $\text{old}X^+ := X^+;$
    for each functional dependency $Y \rightarrow Z$ in F do
        if $X^+ \supseteq Y$ then $X^+ := X^+ \cup Z;$
until $(X^+ = \text{old}X^+);$

1. $X^+ = X$
2. 현재 있는 X^+로 모든 FD를 확인하며 FD의 Left-hand set이 $X+$의 Subset이면 Right-hand (Z)와 X^+을 UNION하여 X^+을 Update한다.
3. (2)번 과정에서 변경된 것이 있다면 (2)번 과정을 다시 반복한다.

Example

$$F = \{\text{Ssn} \rightarrow \text{Ename},$$
$$\text{Pnumber} \rightarrow \{\text{Pname, Plocation}\},$$
$$\{\text{Ssn, Pnumber}\} \rightarrow \text{Hours}\}$$

$\{\text{Ssn}\}^+ = \{\text{Ssn, Ename}\}$
$\{\text{Pnumber}\}^+ = \{\text{Pnumber, Pname, Plocation}\}$
$\{\text{Ssn, Pnumber}\}^+ = \{\text{Ssn, Pnumber, Ename, Pname, Plocation, Hours}\}$